

Abstract

The Morse Code Translator is a cross-platform CLI application developed in C that provides bidirectional translation between English text and Morse code. The system implements a modular architecture comprising seven functional modules including translation engine, audio-visual playback system, history management, settings configuration, clipboard integration, platform abstraction layer, and input validation mechanisms. The application utilizes a 54-entry lookup table for character-to-morse conversion, supports real-time audiovisual feedback during translation, and maintains a circular buffer storing the last 10 translations. The interface employs a structured menu system with comprehensive user interaction features including a first-run setup wizard and runtime configuration management. Implementation leverages the C Standard Library along with platform-specific APIs for Windows, Linux, and macOS compatibility, featuring static memory allocation and session-based data persistence. The project demonstrates effective integration of data structures, algorithms, and cross-platform programming techniques to create a functional and user-friendly morse code translation utility.

List of Figures

Sl.No.	Figure Name	Page
3.1.1	Flowchart of Morse Code Translator	9
5.2.1	OS warning	13
5.2.2	Setup Wizard Welcome	13
5.2.3	Setup Wizard: Sound	14
5.2.4	Setup Wizard: Lighting	14
5.2.5	Setup Wizard: Speed	15
5.2.6	Setup Wizard: Summary	15
5.2.7	Main Menu	15
5.2.8	English To Morse Translation	15
5.2.9	Morse To English Translation	16
5.2.10	Translation Histories	16
5.2.11	Translation History Replay	16
5.2.12	Settings Menu	17
5.2.13	Exiting Program	17

Table of Contents

Abstract.....	i
List of Figures.....	ii
Chapter: 1 Introduction	6-7
1.1. Problem Specification/Statement	
1.2. Objectives	
1.3. Flow of the Project	
1.4. Organization Of Project Report	
Chapter: 2 Background.....	8
2.1. Existing System Analysis	
Chapter: 3 System Analysis & Design	9
3.2.1. Algorithms/Flowchart	
Chapter: 4 Implementation.....	10-12
4.1. Interface Design/Front-End	
4.2. Back-End	
4.3. Modules	
Chapter: 5 User Manual.....	13-17
5.1. System Requirement	
5.1.1. Hardware Requirement	
5.1.2. Software Requirement	
5.2. User Interfaces	
5.2.1 Panel A	
5.2.2 Panel B	
5.2.3. <u>Login Credentials</u>	
Chapter: 6 Conclusion.....	18
6.1. Conclusion	
6.2. Limitation	
6.3. Future Works	
References	19

Chapter 1

INTRODUCTION

1.1. Problem Specification

Communication is essential in modern society, but standard methods can fail in critical situations. Morse code remains valuable in emergencies, military use, aviation, and assistive communication because it is simple and reliable. However, learning and translating Morse code manually is slow and prone to errors.

Many existing Morse code tools are limited, platform-dependent, or lack features such as sound output, visual feedback, translation history, customization, and full two-way translation. This project addresses these gaps by providing a complete, interactive, cross-platform, console-based Morse Code Translator with accurate translation, audio and visual support, history tracking, and customizable settings.

1.2. Objectives

The main objectives of this project are:

- To design and implement a Morse Code Translator using the C programming language
- To translate English text to Morse code accurately
- To translate Morse code back to English text
- To provide sound effects (beeps) for dots and dashes
- To provide visual light effects during Morse playback
- To maintain a translation history with replay functionality
- To allow users to customize settings such as sound, lights, and speed
- To ensure cross-platform compatibility for Windows, Linux, and macOS
- To create a user-friendly, menu-driven console interface

These objectives are evaluated at the end of the project to determine the overall success of the system.

1.3. Scope

The scope of this project includes:

- Development of a console-based application using C
- Support for:
 - English alphabets (A–Z)
 - Numbers (0–9)
 - Common punctuation symbols
- English-to-Morse and Morse-to-English translation
- Sound-based Morse playback using system beeps
- Visual feedback using console background color changes
- Adjustable translation speed (Slow, Medium, Fast)
- Translation history storage and replay
- Clipboard copy support for translated output
- Cross-platform execution on Windows, Linux, and macOS

The project does not include a graphical user interface (GUI) or mobile application support, which may be considered for future enhancement.

1.4. Organization Of Project Report

Chapter 1 – Introduction: introduces the Morse Code Translator project. It explains the problem of manual Morse code translation, outlines the objectives of the system, and defines the scope, including supported features, cross-platform functionality, and key limitations.

Chapter 2 - Background: discusses the background of the project. It includes the analysis of the existing system, its limitations, and the need for the proposed system. This chapter also reviews supporting literature related to Morse code and similar systems.

Chapter 3 – System Analysis & Design: presents the analysis and design of the Morse Code Translator system. It includes the system's flowchart, illustrating the step-by-step process of translation, sound and visual feedback, customization, and history management within the application.

Chapter 4 – Implementation: describes the implementation of the Morse Code Translator system. It covers the CLI-based front-end design with a menu-driven interface, real-time translation display, audio-visual feedback, status messages, and additional features like history and settings. The chapter also details the backend structure, including platform abstraction, data management, translation engine, and control logic, as well as module functionalities for translation, audio-visual playback, history management, settings, clipboard integration, and input validation.

Chapter 5 – User Interface: outlines the hardware and software requirements for the Morse Code Translator system. The hardware requirements emphasize broad compatibility and smooth operation on entry-level systems with minimal dependencies. The software requirements specify the development environment (Code::Blocks recommended), compatible C compilers (GCC, MinGW, MSVC, Clang), C standard (C99 or higher), and cross-platform support for Windows, Linux, and macOS. The chapter also includes illustrative images of the application's user interface.

Chapter 6 - Conclusion: presents the overall conclusion of the project. It summarizes the achievements of the system, highlights the limitations of the current implementation, and suggests possible future enhancements for further development of the project. Thus, each chapter contributes to a clear understanding of the design, implementation, and significance of the Morse Code Translator system.

Chapter 2

BACKGROUND

2.1. Existing System Analysis

Communication has always been a fundamental need of human civilization, and Morse code represents one of the earliest and most reliable techniques for transmitting information over long distances. Traditionally, Morse code communication relied on manual methods such as telegraph keys, printed Morse charts, and human interpretation of sound or light signals. While effective in the past, these systems required significant training, memorization, and continuous practice.

In the modern era, several digital tools and online applications exist for Morse code translation. However, after studying the existing systems, the following limitations were identified:

- Most available translators are web-based and require a continuous internet connection.
- Many systems support only one-way translation (English to Morse) and lack Morse to English decoding.
- Limited support for audio and visual signal simulation, which is essential for learning Morse code practically.
- Absence of customizable speed control, making it difficult for beginners and advanced learners to practice effectively.
- No proper translation history management, which restricts revision and learning from previous attempts.
- Poor error handling and limited support for punctuation and special characters.

As a result, learners face difficulties in understanding the real-world behavior of Morse communication systems. To overcome these drawbacks, a standalone, interactive, and offline-capable solution is required.

The proposed Morse Code Translator system addresses these issues by providing a console-based application developed using the C programming language. The system supports bidirectional translation, adjustable speed control, sound and visual feedback, and maintains a history of translations. This makes the system more efficient, educational, and suitable for both academic and practical use.

Chapter 3

SYSTEM ANALYSIS & DESIGN

3.1. Model & Diagram

3.1.1. Algorithms/Flowchart

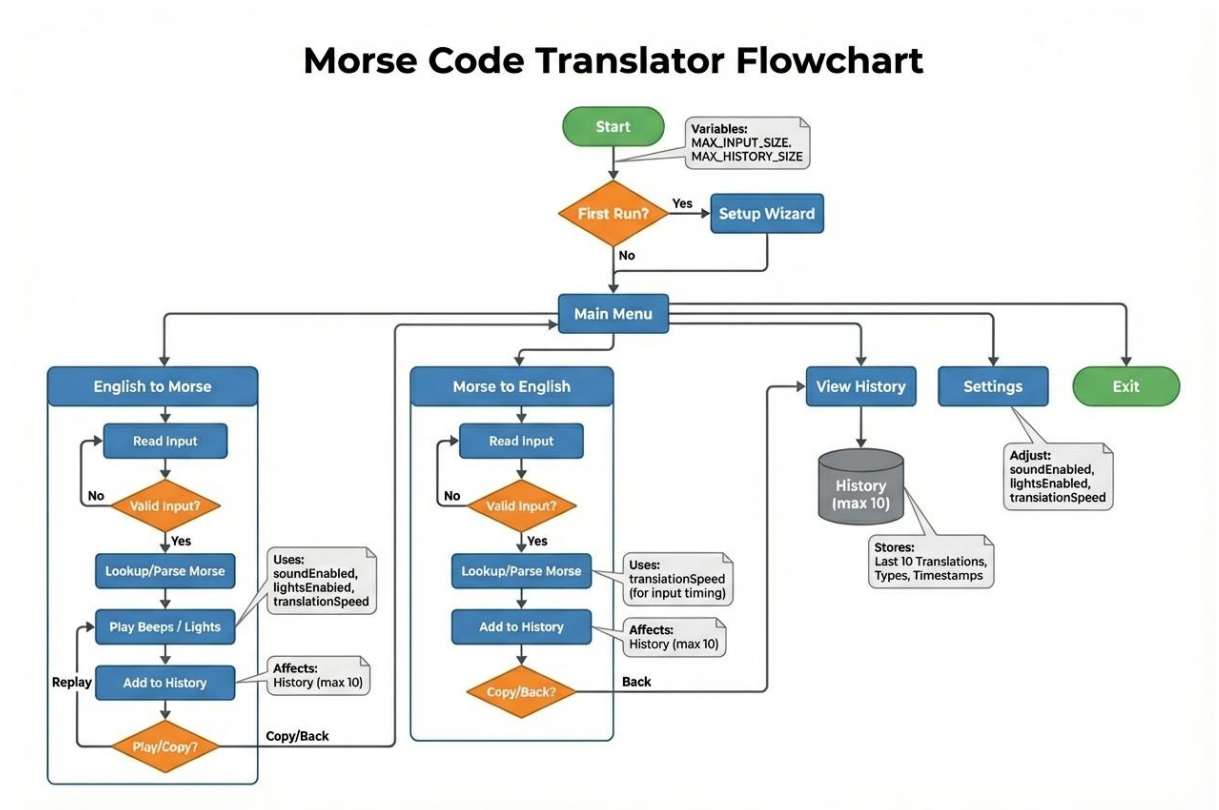


Fig: 3.1.1 : Flowchart of Morse Code Translator

Chapter 4

IMPLEMENTATION

4.1. Interface/Front-End Design

The application uses a **CLI-based interface** focused on simplicity, clarity, and fast interaction. A menu-driven system with real-time feedback and optional audio-visual output improves usability.

- ❖ **Main Menu:** Numbered options for translation (both directions), history, settings, and exit, with consistent layout and clear headers.
- ❖ **Translation Display:**
 - *English* → *Morse*: Word-by-word conversion, pipe separators between characters, real-time playback, and full output display.
 - *Morse* → *English*: Character-by-character decoding with explicit word separators and clean final output.
- ❖ **Visual & Audio Feedback:**
 - Screen flashes (white/black) synchronized with audio beeps during morse playback.
 - Both features can be toggled in settings.
- ❖ **Status Messages:**
 - [OK] for successful actions, [X] for errors.
 - Immediate feedback for all inputs.
- ❖ **Additional Features:**
 - First-run setup wizard
 - Translation history viewer
 - Runtime settings menu
 - Input validation with error reporting

4.2. Interface/Front-End Design

The backend is divided into four layers:

1. Platform Abstraction Layer (OS-specific handling)
2. Data Management Layer (morse tables and history)
3. Translation Engine
4. Control Logic Layer (menu flow and coordination)

Platform-Specific Support

- Cross-platform handling for:
 - Sleep timing (Sleep/usleep)
 - Audio (Beep/terminal bell)
 - Screen control (Windows API / ANSI codes)
 - Clipboard access (Windows API, pbcopy, xclip)

Translation Engine

❖ English to Morse:

- Input parsed character-by-character
- Characters converted to uppercase and matched against a 54-entry morse table
- Spaces converted to / for word separation
- Example: HELLO → -.-. -.-. ---

❖ Morse to English:

- Dot-dash sequences parsed individually
- / interpreted as space
- Unknown codes replaced with ?
- Example: -.-. -.-. --- → HELLO

4.3. Modules/Features

Module Overview

- **Translation-Engine**
Bidirectional translation, 54-character support, real-time playback, and unknown character handling.
- **Audio-Visual-Playback:**
Synchronized sound and screen flashing with speed control and toggles.
- **Translation-History**
Circular buffer storing last 10 translations with replay functionality.
- **Settings-Management**
Sound, lights, speed configuration, and first-run setup wizard.
- **Clipboard-Integration:**
Cross-platform copy-to-clipboard with feedback.
- **Platform-Abstraction-Layer**
Unified API for OS-dependent operations.
- **Input-Validation&Safety**
Buffer protection, length checks, safe string handling, and range validation.

Chapter 5

USER MANUAL

5.1. System Requirement

5.1.1. Hardware Requirement

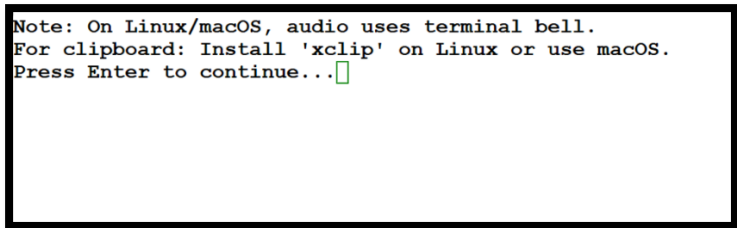
- This solution is highly accessible and compatible with a broad range of hardware specifications.
- Ensures smooth operation even on entry-level systems.
- Maintains high performance with minimal hardware dependency.
- Software Requirement

5.1.2. Software Requirement

- Development Environment Code::Blocks (Recommended)
- Compiler Compatibility: Any standard C compiler (GCC, MinGW, MSVC, Clang)
- Minimum Standard: C99 or higher
- Platform Support: Windows, Linux, macOS

5.2. User Interfaces

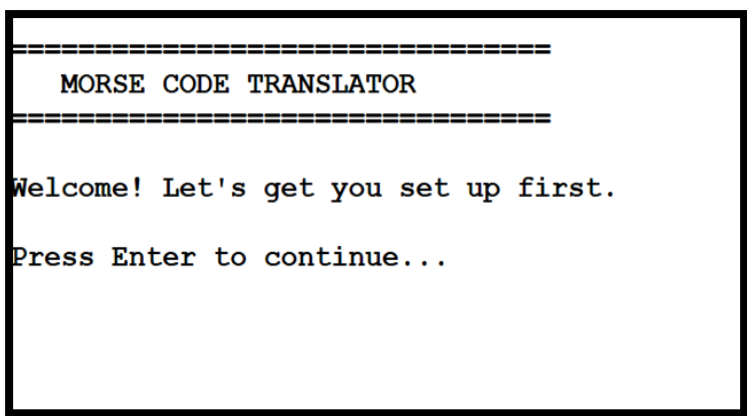
5.2.1. Warning for Other Os



```
Note: On Linux/macOS, audio uses terminal bell.  
For clipboard: Install 'xclip' on Linux or use macOS.  
Press Enter to continue...
```

Fig 5.2.1: OS warning

5.2.2. Setup Wizard



```
=====
MORSE CODE TRANSLATOR
=====

Welcome! Let's get you set up first.

Press Enter to continue...
```

Fig 5.2.2: Setup Wizard Welcome

5.2.3. Setup Wizard: Sound

```
=== Setup: Sound ===  
  
Do you want to enable sound effects?  
(Sound plays beeps for dots and dashes)  
  
1. Yes  
2. No  
  
Enter choice (1-2): █
```

Fig 5.2.3: Setup Wizard: Sound

5.2.4. Setup Wizard: Lighting

```
=== Setup: Visual Effects ===  
  
Do you want to enable visual lights?  
(Shows visual indicators during morse playback)  
  
1. Yes  
2. No  
3. Go back  
  
Enter choice (1-3):
```

Fig 5.2.4: Setup Wizard: Lighting

5.2.5. Setup Wizard: Speed

```
=== Setup: Translation Speed ===  
  
Select translation speed:  
  
1. Slow  
2. Medium  
3. Fast  
4. Go back  
  
Enter choice (1-4):
```

Fig 5.2.5: Setup Wizard: Speed

5.2.6. Setup Wizard: Summary

```
=== Setup Summary ===

Your settings:
  - Sound: OFF
  - Visual Lights: OFF
  - Speed: FAST

1. Continue
2. Go back to change settings

Enter choice (1-2):
```

Fig 5.2.6: Setup Wizard: Summary

5.2.7. Main Menu

```
=====
      MORSE CODE TRANSLATOR
=====

1. English to Morse Code
2. Morse Code to English
3. History (View & Replay)
4. Settings
5. Exit

Enter your choice (1-5): █
```

Fig 5.2.7: Main Menu

5.2.8. English To Morse

```
=== ENGLISH TO MORSE CODE ===
Input: hello

Translation:

[HELLO] = . . . . | . | . - . . | . - . . | - - -

[OK] Translation completed!

Full Output:
.....-.-.-. ---

Copy to clipboard? (1=Yes, 0=No): █
```

Fig 5.2.8: English To Morse Translation

5.2.9. Morse To English

```
=== MORSE CODE TO ENGLISH ===
Enter Morse code (spaces between letters, / for word breaks)
Example: .... . .-... .-... --- / .-- --- .-. .-... -..

Input: .... . .-... .-... ---

Translating: [H] [E] [L] [L] [O]

--- English Translation ---
HELLO

[OK] Translation completed!

Copy to clipboard? (1=Yes, 0=No): █
```

Fig 5.2.9: Morse To English Translation

5.2.10. Translation Histories

```
=== Translation History ===
(Last 2 translations)

1. [English -> Morse] hello
2. [Morse -> English] .... . .-... .-... ---

0. Back to Main Menu

Enter number to view/replay (0-2): █
```

Fig 5.2.10: Translation Histories

5.2.11. Translation History Replay

```
=== Translation #1 ===
Type: English to Morse

Input:
hello

Output:
.... . .-... .-... ---

--- Options ---
1. Replay with sound
2. Copy output to clipboard
3. Back to history
Enter choice (1-3):
```

Fig 5.2.11: Translation History Replay

5.2.12. Settings

```
=== Settings ===  
  
1. Sound: OFF  
2. Visual Lights: OFF  
3. Translation Speed: FAST  
4. Back to Main Menu  
  
Enter your choice (1-4):
```

Fig 5.2.12: Settings Menu

5.2.13. Exiting Program

```
Quitting...  
  
...Program finished with exit code 0  
Press ENTER to exit console.█
```

Fig 5.2.13: Exiting Program

Chapter 6

CONCLUSION

6.1. Conclusion

The Morse Code Translator project has been successfully designed and implemented as a complete, interactive, and offline application using the C programming language. The system effectively translates English text into Morse code and decodes Morse code back into readable English. By incorporating sound effects, visual light simulation, adjustable translation speed, and translation history, the project closely mimics real-world Morse communication behavior. This project demonstrates the practical application of fundamental programming concepts such as structures, arrays, conditional statements, file-independent data handling, and modular function design. It also highlights the importance of user-centered system design by providing clear menus, validation mechanisms, and replay options.

The developed system fulfills all predefined objectives and serves as a valuable learning tool for students interested in communication systems, programming, and digital signal representation

6.2. Limitations

Although the system performs efficiently, certain limitations exist:

- The application is console-based, which limits graphical interaction.
- Sound output depends on the operating system and terminal support.
- Real-time decoding of live Morse signals is not implemented.
- Hardware-level signal transmission (LEDs, buzzers) is not supported.
- Language support is limited to the predefined Morse table.

These limitations do not affect the core functionality but indicate areas for enhancement.

6.3. Future Works

The scope of this project can be extended in several creative and practical directions:

- Development of a graphical user interface (GUI) for improved usability.
- Integration with microcontrollers (Arduino/Raspberry Pi) for real hardware signal transmission.
- Implementation of real-time Morse input detection using keyboard timing or sensors.
- Expansion to mobile and web platforms.
- Support for multiple languages and customizable Morse mappings.

With these future enhancements, the Morse Code Translator can evolve into a powerful educational and communication tool suitable for both academic research and real-world applications.

References

1. Compiler Used for figures: https://www.onlinegdb.com/online_c_compiler
2. Online Morse Code Translator that was used as a reference:
<https://morsecode.world/international/translator.html>